# Victor Mbachu

Senior Full-Stack Engineer & Infrastructure Co-Owner

**GitHub:** @cutewizzy11

**Access Level:** Co-Owner (Admin)

**Reports To:** @Goldokpa (Project Owner)

**Project Duration:** 3 Months

## How This Role Fits You

At Zeta Global you design and run distributed microservice systems handling over 2 million API requests daily with 99.9% uptime across multiple AWS regions - ECS Fargate clusters, RDS Aurora, SNS/SQS messaging, and blue-green CI/CD deployments provisioned via Terraform. You also serve as on-call engineer with a 15-minute average incident resolution time. That is the production engineering standard ClimateVision needs to reach, and you have already built it professionally.

At RWS Global you containerised applications with Docker, deployed across dev, staging, and production environments, led a team of 3 engineers in Agile sprints, and maintained GitHub Actions CI/CD pipelines with TDD coverage. The Docker and deployment ownership on this project - previously unassigned - is a natural fit: you do this as part of your day job, not as a stretch task.

Your stack breadth is the reason you can serve as repository co-owner rather than just a frontend contributor. React, Next.js, Vue, TypeScript, Node.js, PHP/Laravel, Python/Django - you can read and reason about the FastAPI backend, the PyTorch inference pipeline, and the React dashboard with equal confidence. Reviewing PRs across four data scientists requires that range. Your AWS Certified Cloud Practitioner and Professional Scrum Master certifications anchor both the infrastructure ownership and the project coordination function.

Your AI integration experience - GPT-4 and Anthropic API work at RWS Global and PetMe - means you understand the ML serving layer you are wrapping with a frontend. When @edoh-Onuh exports a model and Olufemi builds the inference API, you are not reading foreign code. You have shipped production AI features before. Your two co-authored papers on agentic AI systems show that engagement runs deeper than implementation.

## Your Role on ClimateVision

You own the frontend application, the CI/CD infrastructure, and the Docker/deployment layer. As co-owner you are also the quality gate for all code entering the repository - the one person on the team who can review and reason about every layer of the stack.

### Core Responsibilities - Frontend

- Build the React/TypeScript dashboard with interactive Leaflet map for satellite analysis results
- Create Recharts components for deforestation trends, carbon metrics, and model performance
- Implement api.ts - the fully-typed API client for all FastAPI backend communication
- Build the alert notification panel for real-time deforestation alerts
- Implement responsive TailwindCSS design for desktop and tablet viewports

- Create the deep-dive analysis page with region selector, date range picker, and model comparison

### Core Responsibilities - Infrastructure & CI/CD

- Own the Dockerfile - multi-stage production build for the FastAPI + frontend application
- Own docker-compose.yml - local development stack wiring API, database, and frontend services
- Build and maintain GitHub Actions CI/CD pipelines: lint, type-check, test, and deploy on every PR
- Manage production environment configuration - dev/staging/prod separation and secrets management
- Serve as first responder for production incidents - triage, diagnose, and coordinate resolution

### Sprint Progress - April 2026

- DONE: GitHub Actions CI pipeline (Python flake8 + pytest, frontend npm build)
- DONE: Test scaffolding (tests/ directory with pytest fixtures)
- DONE: Frontend build fixes (case-sensitive import paths)
- DONE: Dependency fixes (removed gdal pip package, added email-validator)
- PENDING: Frontend unit tests with Vitest + React Testing Library
- PENDING: Auth UI - capture X-API-Key in AppContext
- PENDING: WebSocket client for real-time run status
- PENDING: Alert notification UI with severity filters
- PENDING: Mask overlay on map component
- PENDING: Docker Compose for full-stack local dev

### Core Responsibilities - Co-Owner

- Review and merge pull requests from all team members (target: <24 hour turnaround)
- Manage GitHub issues, milestones, project boards, and sprint planning
- Enforce branch protection rules, code quality standards, and API contract consistency
- Manage the release process: version tagging, changelog, and release notes

## Your Codebase Ownership

You are the primary owner of the following files and directories:

```
frontend/                              # PRIMARY OWNER - Entire frontend
  src/
    App.tsx                             # Main application shell
    api.ts                              # Typed API client
    main.tsx                            # Entry point
    styles.css                          # TailwindCSS styles
    components/                         # Component library
      Map.tsx                           # Leaflet map
      ResultsViewer.tsx                 # Prediction results
      Charts.tsx                        # Recharts visualizations
      AlertPanel.tsx                    # Alert notifications
      Settings.tsx                      # User settings
    pages/
```

```
      Dashboard.tsx                      # Main dashboard
      Analysis.tsx                       # Deep analysis view
      History.tsx                        # Run history
  package.json | vite.config.ts | tsconfig.json

Dockerfile                             # PRIMARY OWNER - Multi-stage production build
docker-compose.yml                     # PRIMARY OWNER - Local development stack

.github/workflows/                     # PRIMARY OWNER
  ci.yml                               # Continuous integration
  deploy.yml                           # Deployment pipeline
  tests.yml                            # Test automation

tests/                                 # CO-OWNER (with all DS engineers)
```

## Your 3-Month Delivery Timeline

### MONTH 1: Foundation (Weeks 1-4)

**Week 1-2: Infrastructure & CI/CD**

- Write multi-stage Dockerfile for optimised API + frontend production image
- Build docker-compose.yml wiring FastAPI, SQLite/PostgreSQL, and frontend services locally
- Set up GitHub Actions CI: lint, type-check, pytest, and Vite build on every PR
- Create branch protection rules: require passing CI and 1 review before merging to develop

**Week 3-4: Frontend Architecture & Core Components**

- Configure React Router, Vite, TypeScript strict mode, TailwindCSS, ESLint, and Prettier
- Build Map.tsx - Leaflet map with GeoJSON overlay for deforestation masks
- Implement api.ts - fully-typed API client for all FastAPI endpoints
- Create Dashboard.tsx - main landing page with summary metrics and run status

### MONTH 2: Feature Development (Weeks 5-8)

**Week 5-6: Data Visualisation**

- Build Charts.tsx - Recharts components for deforestation trend lines, bar charts, gauges
- Create ResultsViewer.tsx - segmentation masks overlaid on satellite imagery
- Implement Analysis.tsx - region selector, date picker, model comparison view
- Set up Vitest and React Testing Library - component test coverage from the start

**Week 7-8: Real-Time & Interactivity**

- Build WebSocket integration for live prediction job status updates
- Create AlertPanel.tsx - real-time deforestation alert notification feed
- Implement History.tsx - paginated, filterable list of past analysis runs
- Build Settings.tsx - user preferences and API key management

**MONTH 3: Production Readiness (Weeks 9-12)**

**Week 9-10: Deployment & Environment Config**

- Configure dev/staging/prod environment separation with secrets management
- Set up deployment pipeline to Vercel (frontend) and Docker-based backend hosting
- Implement health monitoring and automated alerting for production incidents
- Performance pass: code splitting, lazy loading, image optimisation, bundle analysis

**Week 11-12: Integration, Testing & Release**

- Full end-to-end integration testing against all backend API endpoints
- Responsive design audit for tablet and large desktop breakpoints
- Accessibility review: keyboard navigation and screen reader compatibility
- Manage v1.0 release: changelog, version tag, release notes, and deployment sign-off

## Your Git Workflow

```
# Create feature branches from develop
git checkout develop
git pull origin develop
git checkout -b feature/frontend-leaflet-map

# Your branch naming convention:
feature/frontend-*     (frontend features)
feature/infra-*        (Docker, CI/CD, deployment)
feature/ci-*           (GitHub Actions changes)
fix/frontend-*         (bug fixes)
release/v*             (release branches)
```

As co-owner, you can merge directly to develop after self-review for frontend-only or infra-only changes. For changes touching shared Python code or API contracts, get a review from @Goldokpa or the relevant module owner.

## Your Key Collaborators

- Olufemi Taiwo (API & Data Quality Lead) - He owns the FastAPI schemas, inference validation, and audit logging. You own the Docker image and deployment pipeline that runs his API. Define the API contract together: endpoint URLs, request/response shapes, auth headers, and error formats.
- @franchaise (Analytics Lead) - His carbon metrics and KPI data feed your dashboard charts. Align on JSON data contracts, refresh intervals, and pagination formats.
- @edoh-Onuh (ML Lead) - Model prediction outputs need to be visualised on the map. Coordinate on GeoJSON output format, confidence score rendering, and how prediction jobs report status via the API.
- @Oshgig (Data Pipeline Lead) - Satellite imagery tile previews on the map may draw on her geospatial utilities. Align on tile formats, coordinate systems, and GeoJSON structures.

## Your Code Pipeline

Your pipeline covers frontend development, Docker orchestration, CI/CD management, and full-stack integration testing.

### Step 1: Environment Setup

```
git clone https://github.com/Climate-Vision/ClimateVision.git
cd ClimateVision

# Backend dependencies
pip install -r requirements.txt

# Frontend dependencies
cd frontend && npm install && cd ..
```

### Step 2: Start Full Local Dev Stack

```
# Option A: Docker Compose (full stack - recommended)
docker-compose up --build
# API:      http://localhost:8000
# Frontend: http://localhost:5173
# MLflow:   http://localhost:5000

# Option B: Run services individually for faster iteration
uvicorn climatevision.api.main:app --reload --port 8000 &
cd frontend && npm run dev
```

### Step 3: Frontend Development Loop

```
cd frontend

# Run linting and type checks
npm run lint
npm run type-check

# Run component tests
npm run test

# Build production bundle and check for errors
npm run build

# Preview production build locally
npm run preview
```

### Step 4: Current CI/CD Configuration

The following .github/workflows/ci.yml is live and runs on every PR to main/develop:

```
name: CI
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
```

```
  python:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.11'}
      - run: sudo apt-get update && sudo apt-get install -y libgl1
      - run: pip install -r requirements.txt && pip install -e .
      - run: flake8 src/ --select=E9,F63,F7,F82
      - run: pytest tests/ -v --tb=short

  frontend:
    runs-on: ubuntu-latest
    defaults: {run: {working-directory: frontend}}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: {node-version: '20', cache: 'npm'}
      - run: npm ci
      - run: npm run build
```

### Step 5: Build & Test Docker Image

```
# Build production Docker image
docker build -t climatevision:latest .

# Run container and verify it starts cleanly
docker run -p 8000:8000 climatevision:latest

# Check all services are healthy inside the container
curl http://localhost:8000/health

# Inspect image size and layers
docker image inspect climatevision:latest | grep Size
```

### Step 6: Run Full CI Checks Locally

```
# Simulate the GitHub Actions CI pipeline before pushing

# 1. Python: lint and tests
flake8 src/ --count --select=E9,F63,F7,F82 --show-source --statistics
flake8 src/ --count --exit-zero --max-complexity=10 --max-line-length=127 --statistics
pytest tests/ -v --tb=short

# 2. Frontend: build
cd frontend && npm run build

# 3. Docker build succeeds
docker-compose build
```

### Step 6: Commit & Push Your Work

```
# Switch to your git identity
source team_docs/switch_user.sh victor

git checkout develop && git pull origin develop
git checkout -b feature/frontend-leaflet-map

git add frontend/src/components/Map.tsx
git add frontend/src/api.ts
git commit -m "feat(frontend): add Leaflet map with GeoJSON deforestation overlay"

git push victor feature/frontend-leaflet-map

# As co-owner: review and merge PRs from the team
# gh pr review <PR_NUMBER> --approve
# gh pr merge <PR_NUMBER> --squash
```